Sp500 5 mins 5/15 breakout Trading Stratergy

John Wallace

June 2025

1 Introduction

Strategy Objective

The objective of this strategy is to consistently generate favorable returns by identifying intraday periods in the S&P 500 that demonstrate statistically significant tendencies toward directional price movement. The framework operates on high-resolution 5-minute interval data and is activated during simple moving average (SMA) crossover events. These crossovers serve as preliminary signals, which are then filtered and validated through two machine learning models to produce a refined entry condition—the "go/no-go" signal.

Building on this foundation, the system also incorporates regime identification to isolate market conditions that exhibit the highest statistical significance for generating meaningful forward returns.

The strategy is designed around a high risk/reward ratio of 1:6, with strict exit criteria: positions are closed either 35 minutes after entry or at the end of the trading session, whichever occurs first. To mitigate forward bias and data leakage, the configuration was finalized using validation data, while performance was tested exclusively on unseen historical data spanning October 2022 to April 2025. All parameters including maximum loss thresholds and starting capital were fixed throughout the evaluation period to ensure the integrity of results.

2 Strategy Rationale

The initial market hypothesis was inspired by prior trading experience using simple moving average (SMA) crossover events. To quantify the potential predictive value of these events, Fisher's Exact Test was applied to historical data spanning from 2008 to 2020. The result yielded a p-value of 0.08824, which does not meet the conventional threshold for statistical significance at the 95% confidence level.

Despite this, the system demonstrated that 53.18% of SMA crossover-up events resulted in price increases over the subsequent seven periods. While this hit rate is not sufficient to formally reject the null hypothesis, it suggests the presence of a modest directional edge that could be exploitable within a systematic trading strategy.As shown in Appendix A

In other words, while the statistical test alone did not provide conclusive evidence, the slight upward bias observed in the historical crossover events motivated further exploration and refinement using machine learning techniques. These were introduced with the aim of improving signal quality and filtering out noise from false crossovers.

Buidling upon my undergraduaute thesis using mahcine leaning tehcniwuqes for asset class preicitno and porfolio modelting , using the ssytems developeemd to increase the statisical signifacnt or confidnce of a postive resultsing trade

3 Data and Feature Engineering

3.1 Feature Selection

Feature selection was carried out in two distinct stages. The first involved compiling a broad set of indicators both academically recognized and widely used in professional trading environments. While the use of technical indicators alone is often criticized for being insufficient or inefficient, due to limited access to alternative datasets, this strategy relied heavily on such indicators to approximate the state of the market at any given moment.

The goal during this phase was to engineer features that captured market volatility, momentum, structural shifts, and time-based patterns. The construction of these features was guided by a combination of academic research, personal trading experience, and discussions with experienced market participants. This phase acknowledged the principle that a model is only as effective as the quality and relevance of its inputs.

The second stage focused on statistical feature evaluation. After compiling over 100 market indicators, the objective was to assess their relationship with future returns specifically, the 7-period forward return. To achieve this, a Light Gradient Boosting Machine (LightGBM) was employed. While more advanced time series models such as LSTM or TFT may provide superior predictive accuracy, they require significantly more computational time and were not the focus at this stage.

The purpose here was not to produce an accurate return forecasting model, but rather to use LightGBM's built-in feature importance measures to rank the predictive relevance of each feature. This helped identify the top 20 features most strongly associated with 7-period forward returns.

These selected features were then refined further, incorporating temporal context (e.g., cyclical time features and lag structures) to support future integration into more complex models such as the Temporal Fusion Transformer (TFT). The result was a focused and compact feature set that served as the foundation for the strategy's predictive modeling pipeline.

To mitigate forward-looking bias and data leakage, the dataset passed through the LightGBM model was a randomly selected subsample comprising approximately 10% of the total data. This deliberate separation ensured that the importance scores were derived independently of the final training or backtest periods, preserving the integrity of the model evaluation process.

3.2 Two Datasets

4 Modeling Approach

During model development, two separate machine learning prediction models were employed, each requiring a distinct dataset tailored to its respective objectives. Broadly speaking, the modeling framework was divided into two categories: **Quantile Prediction** and **Classification**.

The **Quantile Prediction** model focused on forecasting the return after 7 periods following an SMA crossover event. In contrast, the **Classification** model aimed to predict whether a breakout would be successful. A breakout was defined as *successful* if the lowest price over the next 7 periods did **not** fall below 1.1 times the 14-period ATR, and the highest price **did** exceed 1.5 times the 14-period ATR. These thresholds were derived from the baseline crossover success rate being higher than 50%.

In theory, the quantile prediction model was designed to generate numerical estimates for the potential risk and profit bounds of a trade (i.e., the predicted range), while the classification model acted as a final filter to assess the likelihood of breakout success.

However, during empirical testing, it became apparent that while the *quantile prediction intervals* were reasonably accurate in estimating expected ranges, the direct use of the lower and upper bounds as trade execution levels was less meaningful in practice. This introduced a new challenge: was the model still applicable to a trading system?

Upon further analysis, although the ML signals did not meet the standard threshold for statistical significance (e.g., p-value < 0.05), they still demonstrated practical improvements. For instance, the mean return of the ML-generated signals was **more than 7 times** greater than that of the standard SMA crossover signals, and the hit rate was marginally better as well.

Signal Statistics Summary

Metric	**ML Signal**	**SMA Crossover Signal**
Signal Count	1,652	1,863
Mean Return	**0.0788%**	0.0145%
Standard Deviation	1.7907%	0.3539%
Hit Rate	56.30%	54.27%
T-Test (vs 0)	$t = 1.7850,\ p = 0.07445$	$t = 1.7643,\ p = 0.07785$
95% Confidence Interval	$0.0788\% \pm 0.0864\%$	$0.0145\% \pm 0.0161\%$
One-sided p-value	**0.03723**	**0.03892**
Statistical Significance	Not significant	Not significant

Table 1: Comparison of ML signal vs SMA crossover signal performance

Model Types

Two machine learning models were employed during this project:

- **Temporal Fusion Transformer (TFT)**
  Provided by the PyTorch Forecasting library, this model was used for quantile-based regression to predict return intervals.
- **XGBoost Classifier**
  Used for binary classification to predict the success or failure of breakout trades.

Together, these models form the foundation of a dual-signal trading system — one for estimating risk/reward and another for filtering based on predicted success probability.

4.1 TFT Model

To ensure the Temporal Fusion Transformer (TFT) model correctly learned the temporal dynamics of crossover-based trades, a masking strategy was implemented using a `Weight` variable. This allowed the model to ignore forward return predictions at time steps that were not associated with crossover events. In effect, the model focused exclusively on periods of predictive relevance.

To speed up experimentation during the early development phase, hyperparameters were selected by training the model on a substantially reduced subset of the training and validation data. While this approach prioritized speed over accuracy, it is important to note that the final training was conducted on the full dataset with a revised feature structure and the masking strategy fully implemented. As such, while the hyperparameters derived from earlier runs remained effective, this mismatch introduces a potential source of inaccuracy or bias in the final model.

The training cycle incorporated three key features that improved both performance and efficiency:

- **Early Stopping:** Automatically halted training when the validation loss failed to improve over a defined number of epochs, preventing overfitting.
- **ReduceLROnPlateau Scheduler:** Lowered the learning rate when progress plateaued, enabling the model to make smaller, more refined adjustments during late-stage training.
- **Checkpoint Callback:** Continuously saved the best-performing model based on validation metrics. This ensured that even if performance declined in later epochs (within the early stopping window), the best model state was retained.

Together, these training mechanisms supported a stable and generalizable learning process, even under noisy or volatile financial time series data.

4.2 XGBoost Model

The XGBoost model is a highly efficient machine learning algorithm based on gradient-boosted decision trees. Its strength lies in its ability to model non-linear relationships and interact effectively with structured features—making it well-suited for classification tasks like predicting successful breakouts.

Compared to other boosting frameworks (e.g., LightGBM or CatBoost), XGBoost offers finer control over model complexity via regularization parameters, which helps prevent overfitting—especially critical in financial datasets where noise is prevalent.

A comprehensive training process was followed for this model. Hyperparameter tuning was performed using `GridSearchCV`, testing combinations of tree depth, learning rate, subsampling ratios, and regularization terms to optimize model performance. The model's effectiveness was primarily evaluated using the F1 score, and validation was performed on an unseen test set to ensure its robustness in generalizing to new breakout signals.

4.3 Regime Identification

Building upon the baseline setup—where a buy signal was triggered if the TFT model predicted two positive quantiles and the classification model indicated a low probability of breakout failure—the next objective was to identify specific market regimes in which the models were most accurate.

To explore this, combinations of session-based context (e.g., time of day), volatility conditions, and trend regimes were analyzed. The goal was to determine under which conditions the models performed most effectively and to selectively activate trading only during those favorable regimes.

This regime-based analysis was conducted exclusively on the validation dataset to avoid forward-looking bias. While validation data does inherently contain some lookahead risk, it was not used to

confirm the final model's validity. Instead, it served as a sandbox for model development and refinement—particularly in determining when to trigger the model in live or backtested conditions.

5 Backtest Setup

A custom-built backtesting engine was developed to evaluate the effectiveness of trade signals generated by the predictive models. The system was intentionally lightweight and modular, serving as a proof of concept to build confidence in the strategy's real-world applicability.

The backtesting framework was governed by three core parameters:
- `max_risk_dollar` per trade,
- the prediction horizon,
- and the trailing stop delta,

alongside predefined stop-loss and take-profit levels.

The prediction horizon was fixed at **7 periods**, in line with the model's design, which was trained to forecast outcomes precisely 7 steps ahead. To ensure consistency with the model's assumptions and avoid unreliable extrapolation, a hard rule was enforced: all trades were automatically closed after 7 periods, regardless of whether the target or stop had been reached. This design choice also simplified data processing, enabling the system to quickly evaluate trade outcomes by checking whether, and when, the price crossed the stop-loss or take-profit levels within the fixed window.

The backtesting engine computed and recorded a comprehensive set of performance metrics, including:
- Win rate,
- Average return,
- Median return,
- Annualized Sharpe ratio,
- Annualized return,
- Maximum drawdown,
- Total number of trades,
- Total profit and loss (PnL),
- Average PnL per trade.

These metrics provided a detailed understanding of the model's effectiveness and informed decisions about live deployment readiness.

5.1 Trading Optimization

To ensure a more rigorous and statistically grounded approach to selecting trading parameters—rather than relying on arbitrary values—a systematic optimization framework was developed. This involved iterating over various combinations of key risk parameters including:
- the trailing stop delta,
- the stop-loss (SL) and take-profit (TP) levels based on ATR multiples,
- and the maximum risk allowed per trade.

Each configuration was evaluated through a custom backtesting system, and the resulting performance was statistically assessed using the following methods:
- A one-sample **t-test** to determine if the mean return of trades was significantly different from zero.
- A **Monte Carlo simulation** to model potential equity paths based on normally distributed returns, allowing for insight into expected performance variability and distribution of outcomes.

The backtests were run across hundreds of combinations of SL/TP ratios and trailing stop deltas. For each simulation, metrics such as annualized return, Sharpe ratio, maximum drawdown, total PnL, and average PnL per trade were computed and ranked to identify optimal configurations.

This approach provided both quantitative confidence and practical reasoning for the parameter sets used in the final strategy.

5.2 Position Sizing Using the Kelly Criterion

Based on the backtest results and performance metrics, a statistically grounded approach was used to determine the optimal position sizing. The **Kelly Criterion**, a well-known formula in betting and

trading theory, was employed to estimate the ideal fraction of capital to risk per trade. This method provides a numerically optimal bet size to maximize long-term capital growth while accounting for both win probability and payoff ratio.

The classic Kelly formula is defined as:

$$f^* = \frac{bp - q}{b}$$

where:

- f^* is the optimal fraction of capital to wager,
- b is the odds received on the bet (i.e., the reward-to-risk ratio),
- p is the probability of a win,
- $q = 1 - p$ is the probability of a loss.

To remain conservative and reduce the risk of overfitting or overexposure—especially considering the potential for data leakage in the validation set—a fractional Kelly approach was used. Specifically, the position size per trade was set to:

$$f_{\text{trade}} = 0.75 \times f^*$$

This adjusted sizing accounts for estimation error and helps prevent excessive drawdowns, despite the model achieving better performance on unseen test data than on validation data.

6 Results and Evaluation

6.1 TFT Model Performance

To avoid forward-looking bias in performance evaluation, only the validation dataset was used to assess the relative accuracy of the models. However, it is important to note that there is some inherent bias since the validation set was used during model development to guide hyperparameter tuning and architecture selection. Therefore, the performance on the validation set may slightly overstate the model's generalization ability to truly unseen data.

6.1.1 Temporal Fusion Transformer (TFT) Model

The TFT model was trained to generate quantile predictions for future returns, specifically providing 5th, 50th, and 95th percentiles—yielding a 90% confidence interval for each prediction. The evaluation focused on several key metrics:

- The difference between predicted median (q50) and the actual value (Mean Absolute Error),
- The coverage rate of actual values within the predicted 90% interval (q05–q95),
- The average width of the prediction intervals (q95–q05),
- The pinball loss for each quantile, which directly evaluates the sharpness and calibration of the prediction intervals.

The following metrics were derived (see Appendix for detailed explanation of pinball loss):

Metric	**Value**
MAE (q50)	0.2232
Coverage (q05–q95)	89.54%
Mean Interval Width (q95–q05)	1.269
Mean Pinball Loss (q05)	0.0440
Mean Pinball Loss (q50)	0.1116
Mean Pinball Loss (q95)	0.0396

6.1.2 Directional Consensus Accuracy

To aid in filtering signals from the crossover strategy, a relaxed directional consensus rule was applied. A long signal was considered valid if two or more of the predicted quantiles indicated an upward movement. The relaxed directional accuracy, defined as agreement between the predicted direction and actual movement, achieved:

- **Directional Consensus Accuracy:** 54.02%
- **Sample Size:** 1864 predictions

6.1.3 Interval Behavior Analysis

Given the probabilistic nature of the Temporal Fusion Transformer (TFT) model's outputs, a natural next step was to explore whether range-bound trading strategies—such as mean-reversion or options-based straddles—might be effective. To evaluate this, two key questions were considered:

1. How often did the actual return remain within the predicted interval?
2. In cases where a breakout occurred (i.e., the actual return exceeded the predicted bounds), how often did the return re-enter the predicted interval within the same 7-period prediction window?

The results provide strong empirical support for the model's interval reliability and suggest potential use cases beyond simple point forecasting. A detailed breakdown is presented below:

- **Prediction Interval Accuracy:** Out of 12,999 predicted values across all windows, 11,278 (86.76%) remained within their respective predicted intervals. This indicates that the model's quantile outputs are meaningfully calibrated and capture the underlying return distribution well.
- **Breakout Summary:** In 805 cases (43.19% of all windows), the actual return broke out of the predicted interval. Of these:
  - 395 were upper bound breakouts, with 91.90% subsequently returning to the interval within the 7-bar window.
  - 410 were lower bound breakouts, with 88.78% returning to the interval within the same window.
- **Return-to-Range Ratio:** Overall, 727 out of 805 breakouts (90.31%) returned to the predicted range within the prediction window, reinforcing the notion that breakouts are often short-lived and potentially exploitable.
- **Retracement Magnitude:** On average, returns reversed approximately 1.02 percentage points after a breakout. Specifically:
  - Upper breakouts retraced by 1.037%
  - Lower breakouts retraced by 0.997%

These findings demonstrate a strong confidence in the prediction intervals, and suggest that even when breakouts occur, they are typically followed by meaningful mean-reverting behavior. This opens the door for broader interpretations of strategy, including the use of the same TFT model to support volatility-aware or range-bound execution frameworks.

6.2 XGBOOST Model Performance

The XGBoost model was originally designed to classify successful breakouts, defined as trades with a risk-to-reward ratio of 1:1.5 — that is, a stop loss of 1.1 ATR and a target profit of 1.5 ATR or greater. The objective was to predict whether a breakout would reach its target before hitting the stop.

However, as shown in the classification report below, the model demonstrated a stronger ability to identify cases where no successful breakout occurred (class 0), rather than correctly predicting successful breakouts (class 1).

```
Classification Report:
              precision    recall  f1-score   support

         0.0     0.7259    0.6081    0.6618      1337
         1.0     0.3150    0.4398    0.3671       548

    accuracy                         0.5592      1885
   macro avg     0.5205    0.5239    0.5144      1885
weighted avg     0.6064    0.5592    0.5761      1885
```

The model achieved an overall accuracy of 55.9%, with a precision of 72.6% on the negative class (no breakout) and only 31.5% on the positive class (successful breakout). The recall for class 1 was 43.9%, suggesting that while some successful breakouts were detected, the model struggled to generalize well in this imbalanced setting.

These results highlight the inherent challenge in predicting rare and asymmetric events like breakout successes, especially when they occur less frequently than failed or neutral setups. Further improvements may include:

- Rebalancing the dataset or using SMOTE/undersampling techniques
- Optimizing the classification threshold to increase sensitivity to successful breakouts

- Incorporating additional features to capture volatility, structure, or sentiment

These results suggest that the XGBoost model is more effective as a filter rather than a signal generator. Specifically, its relatively high precision and recall on the negative class (no breakout) indicate that it can reliably identify scenarios where trades are less likely to succeed. In practice, this means the model may be better suited to **screen out false breakouts**, thereby improving the overall quality of trades that are taken.

Rather than directly triggering entries, the model could therefore serve as a **risk-reduction filter**, allowing a trading strategy to avoid lower-probability setups while leaving potential breakouts to be confirmed through other complementary signals or models.

6.3 Regime Identification

We performed regime identification using a combination of volatility-based clustering and trend classification. The goal was to segment the market into distinct macro environments ("regimes") and analyze how strategy performance varies across them.

The identified regimes were compared based on key performance metrics, including **Sharpe ratio**, **win rate**, **average and total PnL**, and **maximum drawdown**. The chart below (Figure 1) visualizes the distribution of returns across the different regimes, highlighting how the return distributions differ from the baseline.

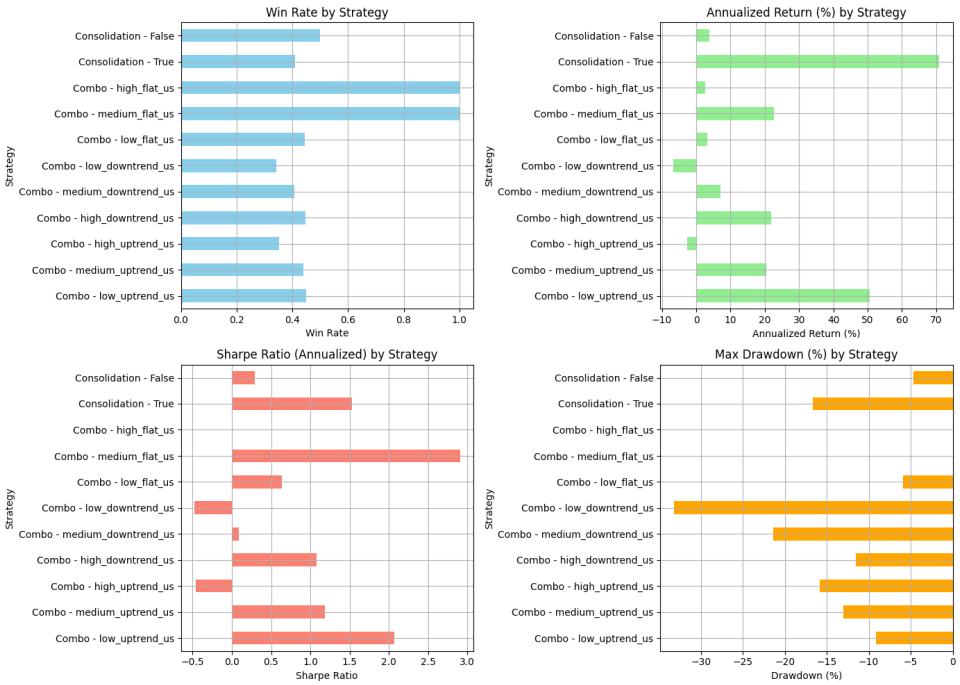

Figure 1: Return distribution by regime

The results from the validation set are summarized in Table 2. These findings were used to construct a more robust model that dynamically adapts to prevailing regimes.

Table 2: Top Regimes Ranked by Sharpe Ratio

Regime	Mean Return	Std Dev	Trades	Win Rate	Avg PnL	Total PnL	Sharpe
High Uptrend US	0.0305	0.2342	93	39.78%	5.85	543.68	0.130
Low Uptrend US	0.0093	0.0860	196	39.80%	4.17	817.28	0.109
Low Downtrend US	0.0092	0.0859	237	40.93%	3.64	863.35	0.107
High Downtrend US	0.0160	0.2158	152	43.42%	3.51	533.86	0.074

The table shows that the *High Uptrend US* regime had the highest Sharpe ratio, although other regimes such as *Low Downtrend US* and *Low Uptrend US* achieved better total PnL due to a higher

number of trades. Notably, the *High Downtrend US* regime exhibited a relatively higher win rate but lower risk-adjusted returns due to increased volatility.

These regime insights form the foundation for a dynamic strategy allocation system that adapts its behavior based on the prevailing market regime.

6.4 Backtesting Results

6.4.1 Statistical Significance

6.5 Model-Based Trade Signal Construction

The foundation of the trading system was built on the observation that the quantile-based predictions produced by the Temporal Fusion Transformer (TFT) model provided reliable directional insights. Specifically, when the model's 5th, 50th, and 95th percentile forecasts skewed positively (i.e., at least two out of three predicted a positive return), it indicated a bullish bias. This bias was used to trigger a preliminary long trade signal.

To enhance robustness and filter out unfavorable trades, a secondary model—an XGBoost classifier—was trained to assess the probability of trade failure. Empirical testing showed that it effectively identified scenarios where the trade was unlikely to succeed. As a result, the final go/no-go decision rule combined both models: a trade was executed only when the TFT quantile forecasts were bullish and the classifier's predicted probability of failure was below 55%.

To evaluate the effectiveness of this combined machine learning (ML) signal, we compared its raw performance (before applying risk management or position sizing) against two benchmarks: a regime-filtered SMA crossover signal and a pure SMA crossover signal. Each signal's forward return distribution over the subsequent 7 bars was statistically tested.

Performance Summary (Regime-Filtered)

- **ML Signal** — Mean Return: 0.1003%, Std: 1.2886%, Trades: 517, Hit Rate: 55.13%
  T-statistic: 1.77, P-value: 0.0773, 95% CI: ±0.1111%
- **SMA Signal** — Mean Return: 0.0060%, Std: 0.2446%, Trades: 1185, Hit Rate: 53.33%
  T-statistic: 0.85, P-value: 0.3981, 95% CI: ±0.0139%

Performance Summary (Unfiltered / All Signals)

- **ML Signal (All)** — Mean Return: 0.1615%, Std: 1.4543%, Trades: 1,523, Hit Rate: 56.07%
  T-statistic: 4.33, P-value: 0.00002, 95% CI: ±0.0730%
- **Pure SMA Crossover Signal** — Mean Return: 0.0087%, Std: 0.2832%, Trades: 1,817, Hit Rate: 52.78%
  T-statistic: 1.30, P-value: 0.1926, 95% CI: ±0.0130%

Table 3: Statistical Comparison: ML Signal vs. SMA Crossover Signal

Metric	ML Signal (All)	SMA Crossover
Signal Count	1,523	1,817
Mean Return	0.1615%	0.0087%
Standard Deviation	1.4543%	0.2832%
Hit Rate	56.07%	52.78%
T-statistic	4.3264	1.3035
P-value	0.00002	0.1926
Statistical Significance	Yes	No
95% Confidence Interval	±0.0730%	±0.0130%

The results indicate that the combined ML signal provides a statistically significant trading edge, with both a substantially higher mean return and a significantly lower p-value compared to the SMA crossover approach. Despite a slightly lower number of trades, the ML signal maintains a higher hit rate and Sharpe-adjusted return.

These findings validate the predictive value of the combined TFT and XGBoost framework, offering a superior foundation for systematic trading strategies when compared to traditional technical rules alone.

6.5.1 Risk Management Analysis

Based on the statistical significance of the signal strength as an indicator of future price increases, the next step was to implement a simplified, yet realistic, risk management framework tailored to the underlying asset's expected volatility. A 14-period rolling Average True Range (ATR) was used to approximate recent price movement magnitude, serving as a foundation for dynamic stop-loss and take-profit levels.

To identify the optimal configuration, several iterations were run with varying stop-loss and take-profit levels expressed as multiples of the ATR, combined with different trailing stop deltas. These combinations were evaluated on the validation dataset, using statistical tools such as t-tests on mean returns, Monte Carlo simulations of equity paths, and bootstrap-based confidence intervals for Sharpe ratios.

The configuration that achieved the **highest annualized return** of **47.34%** with a **Sharpe ratio of 2.84** used a stop-loss of 1.0 ATR, a take-profit of 3.0 ATR, and a trailing stop delta of 1.0. This setup also had a statistically significant t-statistic of 2.59 (p-value = 0.0098), confirming the non-randomness of its return profile. The Monte Carlo simulation showed a mean simulated equity of $547.94 with a standard deviation of $35.28, further illustrating that the strategy outperformed a random walk baseline.

It is important to note that these metrics were derived using the validation set, meaning there is an inherent risk of overfitting. Despite this, the consistency across statistical tests provides preliminary evidence that the strategy's edge may generalize beyond the sample.

6.5.2 Backtesting Results

This backtest was conducted on a dedicated test set, which was never exposed to the model during either the training or validation phases. Using a risk management configuration that limited the maximum dollar loss per trade to $12.25 and starting with an initial capital of $1,000, the system followed an ATR-based strategy with a stop-loss set at 1.0×ATR and a take-profit set at 3.0×ATR, along with a trailing stop.

- **Total Trades:** 1,522
- **Win Rate:** 45.99%
- **Average Return per Trade:** 0.0103%
- **Median Return per Trade:** -0.0324%
- **Sharpe Ratio:** 1.71
- **Annualized Return:** 64.48%
- **Max Drawdown:** -18.20%
- **Total P&L:** $2,435.89
- **Average P&L per Trade:** $1.60

The results indicate that while the system had a relatively modest win rate, it was able to maintain profitability by achieving a high reward-to-risk ratio on successful trades. The Sharpe ratio of 1.71 demonstrates strong risk-adjusted performance, and the annualized return of over 64% suggests consistent compounding. However, a maximum drawdown of 18.20% signals the need for capital buffering or further refinement for risk-sensitive deployment.

A Appendix

B Statistical Evaluation of SMA Crossovers

B.1 Code Implementation

The following Python script was used to evaluate whether SMA crossover-up events are statistically associated with upward price movement 7 periods later. Fisher's Exact Test was selected for this analysis due to its suitability for binary categorical outcomes and small sample independence.

```
import pandas as pd
```

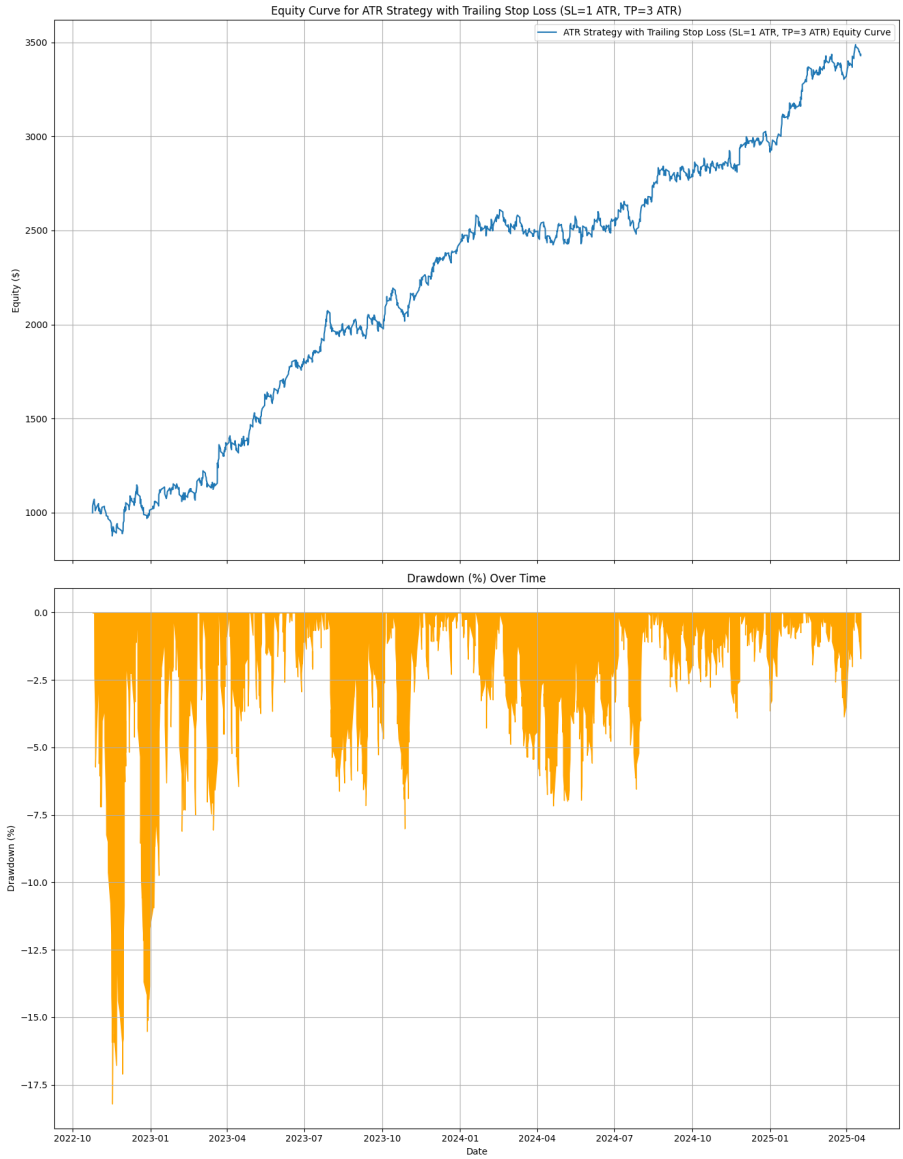

Figure 2: Equity Curve of ATR Strategy on Test Set

```
import numpy as np
from scipy.stats import chi2_contingency, fisher_exact

# Load data
df = pd.read_csv("your_file.csv")
df["time"] = pd.to_datetime(df["time"])
df.set_index("time", inplace=True)

# Calculate SMAs
df["sma1"] = df["Close"].rolling(window=9).mean()
df["sma2"] = df["Close"].rolling(window=21).mean()

# Identify cross-up events
df["cross_up"] = (df["sma1"] > df["sma2"]) & (df["sma1"].shift(1) <= df["sma2"].shift(1)
    )

# Label future direction after 7 periods
df["future_direction"] = np.where(df["Close"].shift(-7) > df["Close"], "up", "down")

# Final filtered dataset
df_valid = df.dropna(subset=["cross_up", "future_direction"])
df_valid["cross_label"] = np.where(df_valid["cross_up"], "cross_up", "no_cross")
```

```python

# Contingency table
contingency = pd.crosstab(df_valid["cross_label"], df_valid["future_direction"])
pval = fisher_exact(contingency)[1]
```

Listing 1: Python Script for SMA Crossover Statistical Test

B.2 Statistical Results

Output Summary

Contingency Table:

```
          future_direction
cross_label   down  up
cross_up      433   492
no_cross     4013  4019
```

Fisher's Exact Test p-value: 0.08824
Conclusion: The relationship between SMA crossovers and future price direction is not statistically significant at the 95% confidence level.
Total Cross-Up Events Analyzed: 925 Successful Cross-Ups (Close(t+7) ¿ Close(t)): 492 Hit Rate: 53.18%
Time Range of Events: 2008-01-02 to 2020-09-30